

ALL-LECTURE CHEATSHEETS

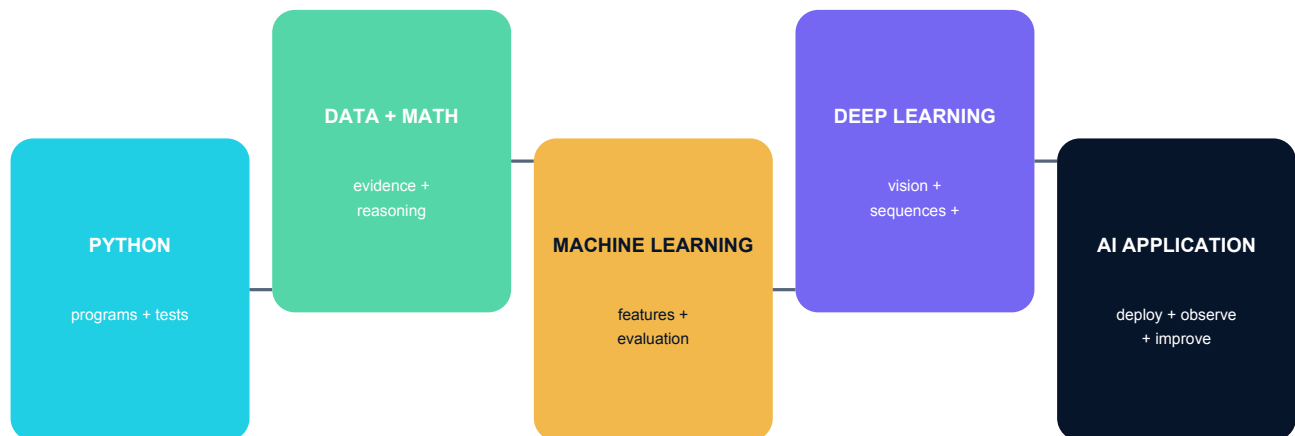
A fast, evidence-centered reference card for every lecture in the complete curriculum.



15 WEEKS	125 LECTURES	FULL STACK
125 lecture cards	Professional resource edition	Learn + build + explain

HOW TO USE THIS BOOK

Review one card before a lecture to prime the durable principle, then return after the lecture to complete the evidence check. The quick action is a command, formula, or engineering sequence-not a substitute for the full lesson.



THE FOUR-PASS REVIEW

- Define - explain the concept without depending on a vendor or library name.
- Implement - create the smallest working example that exposes the mechanism.
- Measure - choose a metric and a negative test before claiming success.
- Explain - connect the result to a user, product, or operating decision.

LECTURE 001 | COURSE INTRODUCTION

Certificate of Completion

Core idea. Place this lecture in the learning journey and connect its vocabulary to an artifact you will build.

Evidence check. Write the prerequisite, expected output, one question, and the next lecture that answers it.

Quick action. `concept -> example -> artifact -> evidence`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 002 | COURSE INTRODUCTION

Introduction to Full-Stack AI Engineer: Python, ML, Deep Learning & GenAI

Core idea. Place this lecture in the learning journey and connect its vocabulary to an artifact you will build.

Evidence check. Write the prerequisite, expected output, one question, and the next lecture that answers it.

Quick action. `concept -> example -> artifact -> evidence`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 003 | COURSE INTRODUCTION

50 Essential AI Concepts: A Comprehensive Journey

Core idea. Place this lecture in the learning journey and connect its vocabulary to an artifact you will build.

Evidence check. Write the prerequisite, expected output, one question, and the next lecture that answers it.

Quick action. `concept -> example -> artifact -> evidence`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 004 | COURSE INTRODUCTION

Quick Quiz with Answers

Core idea. Place this lecture in the learning journey and connect its vocabulary to an artifact you will build.

Evidence check. Write the prerequisite, expected output, one question, and the next lecture that answers it.

Quick action. `concept -> example -> artifact -> evidence`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 005 | COURSE INTRODUCTION

Resources for the Course - Slides and Code Files

Core idea. Place this lecture in the learning journey and connect its vocabulary to an artifact you will build.

Evidence check. Write the prerequisite, expected output, one question, and the next lecture that answers it.

Quick action. `concept -> example -> artifact -> evidence`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 006 | WEEK 01

Introduction to Week 1 Python Programming Basics

Core idea. Place this lecture in the learning journey and connect its vocabulary to an artifact you will build.

Evidence check. Write the prerequisite, expected output, one question, and the next lecture that answers it.

Quick action. `concept -> example -> artifact -> evidence`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 007 | WEEK 01

Day 1: Introduction to Python and Development Setup

Core idea. Write clear programs from small functions, explicit data structures, predictable control flow, and safe file operations.

Evidence check. Run from a clean virtual environment; test normal, boundary, and invalid inputs; use readable names and small functions.

Quick action. `python -m venv .venv && python -m pytest`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 008 | WEEK 01

Day 2: Control Flow in Python

Core idea. Write clear programs from small functions, explicit data structures, predictable control flow, and safe file operations.

Evidence check. Run from a clean virtual environment; test normal, boundary, and invalid inputs; use readable names and small functions.

Quick action. `python -m venv .venv && python -m pytest`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 009 | WEEK 01

Day 3: Functions and Modules

Core idea. Write clear programs from small functions, explicit data structures, predictable control flow, and safe file operations.

Evidence check. Run from a clean virtual environment; test normal, boundary, and invalid inputs; use readable names and small functions.

Quick action. `python -m venv .venv && python -m pytest`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 010 | WEEK 01

Day 4: Data Structures (Lists, Tuples, Dictionaries, Sets)

Core idea. Write clear programs from small functions, explicit data structures, predictable control flow, and safe file operations.

Evidence check. Run from a clean virtual environment; test normal, boundary, and invalid inputs; use readable names and small functions.

Quick action. `python -m venv .venv && python -m pytest`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 011 | WEEK 01

Day 5: Working with Strings

Core idea. Write clear programs from small functions, explicit data structures, predictable control flow, and safe file operations.

Evidence check. Run from a clean virtual environment; test normal, boundary, and invalid inputs; use readable names and small functions.

Quick action. `python -m venv .venv && python -m pytest`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 012 | WEEK 01

Day 6: File Handling

Core idea. Write clear programs from small functions, explicit data structures, predictable control flow, and safe file operations.

Evidence check. Run from a clean virtual environment; test normal, boundary, and invalid inputs; use readable names and small functions.

Quick action. `python -m venv .venv && python -m pytest`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 013 | WEEK 01

Day 7: Pythonic Code and Project Work

Core idea. Write clear programs from small functions, explicit data structures, predictable control flow, and safe file operations.

Evidence check. Run from a clean virtual environment; test normal, boundary, and invalid inputs; use readable names and small functions.

Quick action. `python -m venv .venv && python -m pytest`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 014 | WEEK 02

Introduction to Week 2 Data Science Essentials

Core idea. Place this lecture in the learning journey and connect its vocabulary to an artifact you will build.

Evidence check. Write the prerequisite, expected output, one question, and the next lecture that answers it.

Quick action. `concept -> example -> artifact -> evidence`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 015 | WEEK 02

Day 1: Introduction to NumPy for Numerical Computing

Core idea. Use homogeneous n-dimensional arrays and vectorized operations; reason about shape, dtype, broadcasting, and copies.

Evidence check. Print shape and dtype, assert dimensional assumptions, compare vectorized output with a small hand-worked case, and inspect memory use.

Quick action. `assert x.ndim == 2; print(x.shape, x.dtype)`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 016 | WEEK 02

Day 2: Advanced NumPy Operations

Core idea. Use homogeneous n-dimensional arrays and vectorized operations; reason about shape, dtype, broadcasting, and copies.

Evidence check. Print shape and dtype, assert dimensional assumptions, compare vectorized output with a small hand-worked case, and inspect memory use.

Quick action. `assert x.ndim == 2; print(x.shape, x.dtype)`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 017 | WEEK 02

Day 3: Introduction to Pandas for Data Manipulation

Core idea. Treat a DataFrame as a typed table whose schema, missingness, keys, and transformations must remain explainable.

Evidence check. Profile rows, dtypes, nulls, duplicates, ranges, keys, and join cardinality before modeling.

Quick action. `df.info(); df.isna().mean().sort_values()`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 018 | WEEK 02

Day 4: Data Cleaning and Preparation with Pandas

Core idea. Treat a DataFrame as a typed table whose schema, missingness, keys, and transformations must remain explainable.

Evidence check. Profile rows, dtypes, nulls, duplicates, ranges, keys, and join cardinality before modeling.

Quick action. `df.info(); df.isna().mean().sort_values()`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 019 | WEEK 02

Day 5: Data Aggregation and Grouping in Pandas

Core idea. Treat a DataFrame as a typed table whose schema, missingness, keys, and transformations must remain explainable.

Evidence check. Profile rows, dtypes, nulls, duplicates, ranges, keys, and join cardinality before modeling.

Quick action. `df.info(); df.isna().mean().sort_values()`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 020 | WEEK 02

Day 6: Data Visualization with Matplotlib and Seaborn

Core idea. Use plots to test questions about distributions, relationships, groups, missingness, and anomalies without hiding scale or uncertainty.

Evidence check. Label axes and units, show sample size, inspect segments, preserve an unmodified view, and write the decision each plot supports.

Quick action. `question -> plot -> pattern -> caveat -> decision`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 021 | WEEK 02

Day 7: Exploratory Data Analysis (EDA) Project

Core idea. Use plots to test questions about distributions, relationships, groups, missingness, and anomalies without hiding scale or uncertainty.

Evidence check. Label axes and units, show sample size, inspect segments, preserve an unmodified view, and write the decision each plot supports.

Quick action. `question -> plot -> pattern -> caveat -> decision`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 022 | WEEK 03

Introduction to Week 3 Mathematics for Machine Learning

Core idea. Place this lecture in the learning journey and connect its vocabulary to an artifact you will build.

Evidence check. Write the prerequisite, expected output, one question, and the next lecture that answers it.

Quick action. `concept -> example -> artifact -> evidence`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 023 | WEEK 03

Day 1: Linear Algebra Fundamentals

Core idea. Connect vectors, matrices, derivatives, and optimization to model computation rather than manipulating symbols in isolation.

Evidence check. Check dimensions, compute a tiny example by hand, compare analytical and numerical gradients, and state the geometric meaning.

Quick action. `relative_error = abs(g_num-g_ref) / (abs(g_ref)+1e-8)`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 024 | WEEK 03

Day 2: Advanced Linear Algebra Concepts

Core idea. Connect vectors, matrices, derivatives, and optimization to model computation rather than manipulating symbols in isolation.

Evidence check. Check dimensions, compute a tiny example by hand, compare analytical and numerical gradients, and state the geometric meaning.

Quick action. `relative_error = abs(g_num-g_ref) / (abs(g_ref)+1e-8)`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 025 | WEEK 03

Day 3: Calculus for Machine Learning (Derivatives)

Core idea. Connect vectors, matrices, derivatives, and optimization to model computation rather than manipulating symbols in isolation.

Evidence check. Check dimensions, compute a tiny example by hand, compare analytical and numerical gradients, and state the geometric meaning.

Quick action. `relative_error = abs(g_num-g_ref) / (abs(g_ref)+1e-8)`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 026 | WEEK 03

Day 4: Calculus for Machine Learning (Integrals and Optimization)

Core idea. Connect vectors, matrices, derivatives, and optimization to model computation rather than manipulating symbols in isolation.

Evidence check. Check dimensions, compute a tiny example by hand, compare analytical and numerical gradients, and state the geometric meaning.

Quick action. `relative_error = abs(g_num-g_ref) / (abs(g_ref)+1e-8)`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 027 | WEEK 03

Day 5: Probability Theory and Distributions

Core idea. Quantify uncertainty from an explicit data-generating process, assumptions, estimator, and decision threshold.

Evidence check. Report sample definition, assumptions, interval or effect size, multiplicity, practical significance, and limits on causal claims.

Quick action. `question -> estimand -> assumptions -> interval -> decision`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 028 | WEEK 03

Day 6: Statistics Fundamentals

Core idea. Quantify uncertainty from an explicit data-generating process, assumptions, estimator, and decision threshold.

Evidence check. Report sample definition, assumptions, interval or effect size, multiplicity, practical significance, and limits on causal claims.

Quick action. `question -> estimand -> assumptions -> interval -> decision`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 029 | WEEK 03

Day 7: Math-Driven Mini Project - Linear Regression from Scratch

Core idea. Model a continuous target, starting with a baseline and checking functional form, residuals, leakage, and regularization.

Evidence check. Report MAE/RMSE in domain units, compare residuals by segment, inspect extrapolation, and keep the final test set untouched.

Quick action. `baseline -> fit -> residuals -> cross-validate -> test once`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 030 | WEEK 04

Introduction to Week 4 Probability and Statistics for Machine Learning

Core idea. Place this lecture in the learning journey and connect its vocabulary to an artifact you will build.

Evidence check. Write the prerequisite, expected output, one question, and the next lecture that answers it.

Quick action. `concept -> example -> artifact -> evidence`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 031 | WEEK 04

Day 1: Probability Theory and Random Variables

Core idea. Quantify uncertainty from an explicit data-generating process, assumptions, estimator, and decision threshold.

Evidence check. Report sample definition, assumptions, interval or effect size, multiplicity, practical significance, and limits on causal claims.

Quick action. question -> estimand -> assumptions -> interval -> decision

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 032 | WEEK 04

Day 2: Probability Distributions in Machine Learning

Core idea. Quantify uncertainty from an explicit data-generating process, assumptions, estimator, and decision threshold.

Evidence check. Report sample definition, assumptions, interval or effect size, multiplicity, practical significance, and limits on causal claims.

Quick action. question -> estimand -> assumptions -> interval -> decision

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 033 | WEEK 04

Day 3: Statistical Inference - Estimation and Confidence Intervals

Core idea. Quantify uncertainty from an explicit data-generating process, assumptions, estimator, and decision threshold.

Evidence check. Report sample definition, assumptions, interval or effect size, multiplicity, practical significance, and limits on causal claims.

Quick action. question -> estimand -> assumptions -> interval -> decision

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 034 | WEEK 04

Day 4: Hypothesis Testing and P-Values

Core idea. Quantify uncertainty from an explicit data-generating process, assumptions, estimator, and decision threshold.

Evidence check. Report sample definition, assumptions, interval or effect size, multiplicity, practical significance, and limits on causal claims.

Quick action. question -> estimand -> assumptions -> interval -> decision

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 035 | WEEK 04

Day 5: Types of Hypothesis Tests

Core idea. Quantify uncertainty from an explicit data-generating process, assumptions, estimator, and decision threshold.

Evidence check. Report sample definition, assumptions, interval or effect size, multiplicity, practical significance, and limits on causal claims.

Quick action. question -> estimand -> assumptions -> interval -> decision

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 036 | WEEK 04

Day 6: Correlation and Regression Analysis

Core idea. Quantify uncertainty from an explicit data-generating process, assumptions, estimator, and decision threshold.

Evidence check. Report sample definition, assumptions, interval or effect size, multiplicity, practical significance, and limits on causal claims.

Quick action. question -> estimand -> assumptions -> interval -> decision

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 037 | WEEK 04

Day 7: Statistical Analysis Project - Analyzing Real-World Data

Core idea. Quantify uncertainty from an explicit data-generating process, assumptions, estimator, and decision threshold.

Evidence check. Report sample definition, assumptions, interval or effect size, multiplicity, practical significance, and limits on causal claims.

Quick action. `question -> estimand -> assumptions -> interval -> decision`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 038 | WEEK 05

Introduction to Week 5 Introduction to Machine Learning

Core idea. Place this lecture in the learning journey and connect its vocabulary to an artifact you will build.

Evidence check. Write the prerequisite, expected output, one question, and the next lecture that answers it.

Quick action. `concept -> example -> artifact -> evidence`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 039 | WEEK 05

Day 1: Machine Learning Basics and Terminology

Core idea. Translate the lecture into a precise concept, a small implementation, and evidence that distinguishes success from failure.

Evidence check. State inputs, assumptions, output contract, metric, failure case, version information, and the next decision.

Quick action. `define -> build -> test -> measure -> explain`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 040 | WEEK 05

Day 2: Introduction to Supervised Learning and Regression Models

Core idea. Model a continuous target, starting with a baseline and checking functional form, residuals, leakage, and regularization.

Evidence check. Report MAE/RMSE in domain units, compare residuals by segment, inspect extrapolation, and keep the final test set untouched.

Quick action. `baseline -> fit -> residuals -> cross-validate -> test once`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 041 | WEEK 05

Day 3: Advanced Regression Models - Polynomial Regression and Regularization

Core idea. Model a continuous target, starting with a baseline and checking functional form, residuals, leakage, and regularization.

Evidence check. Report MAE/RMSE in domain units, compare residuals by segment, inspect extrapolation, and keep the final test set untouched.

Quick action. `baseline -> fit -> residuals -> cross-validate -> test once`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 042 | WEEK 05

Day 4: Introduction to Classification and Logistic Regression

Core idea. Model a continuous target, starting with a baseline and checking functional form, residuals, leakage, and regularization.

Evidence check. Report MAE/RMSE in domain units, compare residuals by segment, inspect extrapolation, and keep the final test set untouched.

Quick action. `baseline -> fit -> residuals -> cross-validate -> test once`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 043 | WEEK 05

Day 5: Model Evaluation and Cross-Validation

Core idea. Learn a mapping from labeled examples and judge it with a split strategy and metric that reflect deployment costs.

Evidence check. Preserve an untouched test set; report confusion matrix, threshold-dependent metrics, calibration, segments, and uncertainty.

Quick action. `train -> validate/CV -> choose threshold -> test once`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 044 | WEEK 05

Day 6: k-Nearest Neighbors (k-NN) Algorithm

Core idea. Learn a mapping from labeled examples and judge it with a split strategy and metric that reflect deployment costs.

Evidence check. Preserve an untouched test set; report confusion matrix, threshold-dependent metrics, calibration, segments, and uncertainty.

Quick action. `train -> validate/CV -> choose threshold -> test once`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 045 | WEEK 05

Day 7: Supervised Learning Mini Project

Core idea. Learn a mapping from labeled examples and judge it with a split strategy and metric that reflect deployment costs.

Evidence check. Preserve an untouched test set; report confusion matrix, threshold-dependent metrics, calibration, segments, and uncertainty.

Quick action. `train -> validate/CV -> choose threshold -> test once`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 046 | WEEK 06

Introduction to Week 6 Feature Engineering and Model Evaluation

Core idea. Place this lecture in the learning journey and connect its vocabulary to an artifact you will build.

Evidence check. Write the prerequisite, expected output, one question, and the next lecture that answers it.

Quick action. concept -> example -> artifact -> evidence

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 047 | WEEK 06

Day 1: Introduction to Feature Engineering

Core idea. Transform raw inputs into stable signals while preventing leakage and preserving the same contract during inference.

Evidence check. Fit preprocessing on training folds only; test unseen categories, missing values, feature order, and schema at prediction time.

Quick action. `Pipeline([('prep', transformer), ('model', estimator)])`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 048 | WEEK 06

Day 2: Data Scaling and Normalization

Core idea. Transform raw inputs into stable signals while preventing leakage and preserving the same contract during inference.

Evidence check. Fit preprocessing on training folds only; test unseen categories, missing values, feature order, and schema at prediction time.

Quick action. `Pipeline([('prep', transformer), ('model', estimator)])`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 049 | WEEK 06

Day 3: Encoding Categorical Variables

Core idea. Transform raw inputs into stable signals while preventing leakage and preserving the same contract during inference.

Evidence check. Fit preprocessing on training folds only; test unseen categories, missing values, feature order, and schema at prediction time.

Quick action. `Pipeline([('prep', transformer), ('model', estimator)])`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 050 | WEEK 06

Day 4: Feature Selection Techniques

Core idea. Transform raw inputs into stable signals while preventing leakage and preserving the same contract during inference.

Evidence check. Fit preprocessing on training folds only; test unseen categories, missing values, feature order, and schema at prediction time.

Quick action. `Pipeline([('prep', transformer), ('model', estimator)])`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 051 | WEEK 06

Day 5: Creating and Transforming Features

Core idea. Transform raw inputs into stable signals while preventing leakage and preserving the same contract during inference.

Evidence check. Fit preprocessing on training folds only; test unseen categories, missing values, feature order, and schema at prediction time.

Quick action. `Pipeline([('prep', transformer), ('model', estimator)])`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 052 | WEEK 06

Day 6: Model Evaluation Techniques

Core idea. Learn a mapping from labeled examples and judge it with a split strategy and metric that reflect deployment costs.

Evidence check. Preserve an untouched test set; report confusion matrix, threshold-dependent metrics, calibration, segments, and uncertainty.

Quick action. `train -> validate/CV -> choose threshold -> test once`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 053 | WEEK 06

Day 7: Cross-Validation and Hyperparameter Tuning

Core idea. Search a bounded parameter space with a leakage-safe objective, explicit budget, and reproducible cross-validation.

Evidence check. Record distributions, folds, seed, trials, failures, runtime, best score uncertainty, and untouched final test performance.

Quick action. `search.fit(X_train, y_train); search.best_params_`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 054 | WEEK 07

Introduction to Week 7 Advanced Machine Learning Algorithms

Core idea. Place this lecture in the learning journey and connect its vocabulary to an artifact you will build.

Evidence check. Write the prerequisite, expected output, one question, and the next lecture that answers it.

Quick action. `concept -> example -> artifact -> evidence`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 055 | WEEK 07

Day 1: Introduction to Ensemble Learning

Core idea. Combine diverse learners to reduce variance or sequentially correct errors, accepting greater tuning and serving complexity.

Evidence check. Compare with a simple baseline using identical folds; report generalization, calibration, latency, model size, and feature behavior.

Quick action. `baseline -> forest -> boosting -> cost/quality decision`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 056 | WEEK 07

Day 2: Bagging and Random Forests

Core idea. Combine diverse learners to reduce variance or sequentially correct errors, accepting greater tuning and serving complexity.

Evidence check. Compare with a simple baseline using identical folds; report generalization, calibration, latency, model size, and feature behavior.

Quick action. `baseline -> forest -> boosting -> cost/quality decision`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 057 | WEEK 07

Day 3: Boosting and Gradient Boosting

Core idea. Combine diverse learners to reduce variance or sequentially correct errors, accepting greater tuning and serving complexity.

Evidence check. Compare with a simple baseline using identical folds; report generalization, calibration, latency, model size, and feature behavior.

Quick action. `baseline -> forest -> boosting -> cost/quality decision`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 058 | WEEK 07

Day 4: Introduction to XGBoost

Core idea. Combine diverse learners to reduce variance or sequentially correct errors, accepting greater tuning and serving complexity.

Evidence check. Compare with a simple baseline using identical folds; report generalization, calibration, latency, model size, and feature behavior.

Quick action. `baseline -> forest -> boosting -> cost/quality decision`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 059 | WEEK 07

Day 5: LightGBM and CatBoost

Core idea. Combine diverse learners to reduce variance or sequentially correct errors, accepting greater tuning and serving complexity.

Evidence check. Compare with a simple baseline using identical folds; report generalization, calibration, latency, model size, and feature behavior.

Quick action. `baseline -> forest -> boosting -> cost/quality decision`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 060 | WEEK 07

Day 6: Handling Imbalanced Data

Core idea. Match sampling, weighting, thresholds, and metrics to rare-class costs without contaminating validation data.

Evidence check. Use stratified splits, resample training data only, inspect precision-recall and calibration, and test relevant segments.

Quick action. `report prevalence + PR-AUC + recall@precision + threshold`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 061 | WEEK 07

Day 7: Ensemble Learning Project - Comparing Models on a Real Dataset

Core idea. Combine diverse learners to reduce variance or sequentially correct errors, accepting greater tuning and serving complexity.

Evidence check. Compare with a simple baseline using identical folds; report generalization, calibration, latency, model size, and feature behavior.

Quick action. `baseline -> forest -> boosting -> cost/quality decision`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 062 | WEEK 08

Introduction to Week 8 Model Tuning and Optimization

Core idea. Place this lecture in the learning journey and connect its vocabulary to an artifact you will build.

Evidence check. Write the prerequisite, expected output, one question, and the next lecture that answers it.

Quick action. `concept -> example -> artifact -> evidence`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 063 | WEEK 08

Day 1: Introduction to Hyperparameter Tuning

Core idea. Search a bounded parameter space with a leakage-safe objective, explicit budget, and reproducible cross-validation.

Evidence check. Record distributions, folds, seed, trials, failures, runtime, best score uncertainty, and untouched final test performance.

Quick action. `search.fit(X_train, y_train); search.best_params_`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 064 | WEEK 08

Day 2: Grid Search and Random Search

Core idea. Search a bounded parameter space with a leakage-safe objective, explicit budget, and reproducible cross-validation.

Evidence check. Record distributions, folds, seed, trials, failures, runtime, best score uncertainty, and untouched final test performance.

Quick action. `search.fit(X_train, y_train); search.best_params_`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 065 | WEEK 08

Day 3: Advanced Hyperparameter Tuning with Bayesian Optimization

Core idea. Search a bounded parameter space with a leakage-safe objective, explicit budget, and reproducible cross-validation.

Evidence check. Record distributions, folds, seed, trials, failures, runtime, best score uncertainty, and untouched final test performance.

Quick action. `search.fit(X_train, y_train); search.best_params_`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 066 | WEEK 08

Day 4: Regularization Techniques for Model Optimization

Core idea. Translate the lecture into a precise concept, a small implementation, and evidence that distinguishes success from failure.

Evidence check. State inputs, assumptions, output contract, metric, failure case, version information, and the next decision.

Quick action. `define -> build -> test -> measure -> explain`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 067 | WEEK 08

Day 5: Cross-Validation and Model Evaluation Techniques

Core idea. Learn a mapping from labeled examples and judge it with a split strategy and metric that reflect deployment costs.

Evidence check. Preserve an untouched test set; report confusion matrix, threshold-dependent metrics, calibration, segments, and uncertainty.

Quick action. `train -> validate/CV -> choose threshold -> test once`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 068 | WEEK 08

Day 6: Automated Hyperparameter Tuning with GridSearchCV and RandomizedSearchCV

Core idea. Search a bounded parameter space with a leakage-safe objective, explicit budget, and reproducible cross-validation.

Evidence check. Record distributions, folds, seed, trials, failures, runtime, best score uncertainty, and untouched final test performance.

Quick action. `search.fit(X_train, y_train); search.best_params_`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 069 | WEEK 08

Day 7: Optimization Project - Building and Tuning a Final Model

Core idea. Translate the lecture into a precise concept, a small implementation, and evidence that distinguishes success from failure.

Evidence check. State inputs, assumptions, output contract, metric, failure case, version information, and the next decision.

Quick action. `define -> build -> test -> measure -> explain`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 070 | WEEK 09

Introduction to Week 9 Neural Networks and Deep Learning Fundamentals

Core idea. Place this lecture in the learning journey and connect its vocabulary to an artifact you will build.

Evidence check. Write the prerequisite, expected output, one question, and the next lecture that answers it.

Quick action. `concept -> example -> artifact -> evidence`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 071 | WEEK 09

Day 1: Introduction to Deep Learning and Neural Networks

Core idea. A neural network composes differentiable layers; backpropagation turns loss sensitivity into gradients for an optimizer update.

Evidence check. Assert tensor shapes, overfit a tiny batch, inspect loss and gradients, separate train/eval behavior, and save the best checkpoint.

Quick action. `zero_grad -> forward -> loss -> backward -> step`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 072 | WEEK 09

Day 2: Forward Propagation and Activation Functions

Core idea. A neural network composes differentiable layers; backpropagation turns loss sensitivity into gradients for an optimizer update.

Evidence check. Assert tensor shapes, overfit a tiny batch, inspect loss and gradients, separate train/eval behavior, and save the best checkpoint.

Quick action. `zero_grad -> forward -> loss -> backward -> step`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 073 | WEEK 09

Day 3: Loss Functions and Backpropagation

Core idea. A neural network composes differentiable layers; backpropagation turns loss sensitivity into gradients for an optimizer update.

Evidence check. Assert tensor shapes, overfit a tiny batch, inspect loss and gradients, separate train/eval behavior, and save the best checkpoint.

Quick action. `zero_grad -> forward -> loss -> backward -> step`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 074 | WEEK 09

Day 4: Gradient Descent and Optimization Techniques

Core idea. A neural network composes differentiable layers; backpropagation turns loss sensitivity into gradients for an optimizer update.

Evidence check. Assert tensor shapes, overfit a tiny batch, inspect loss and gradients, separate train/eval behavior, and save the best checkpoint.

Quick action. `zero_grad -> forward -> loss -> backward -> step`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 075 | WEEK 09

Day 5: Building Neural Networks with TensorFlow and Keras

Core idea. A neural network composes differentiable layers; backpropagation turns loss sensitivity into gradients for an optimizer update.

Evidence check. Assert tensor shapes, overfit a tiny batch, inspect loss and gradients, separate train/eval behavior, and save the best checkpoint.

Quick action. `zero_grad -> forward -> loss -> backward -> step`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 076 | WEEK 09

Day 6: Building Neural Networks with PyTorch

Core idea. A neural network composes differentiable layers; backpropagation turns loss sensitivity into gradients for an optimizer update.

Evidence check. Assert tensor shapes, overfit a tiny batch, inspect loss and gradients, separate train/eval behavior, and save the best checkpoint.

Quick action. `zero_grad -> forward -> loss -> backward -> step`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 077 | WEEK 09

Day 7: Neural Network Project - Image Classification on CIFAR-10

Core idea. Learn a mapping from labeled examples and judge it with a split strategy and metric that reflect deployment costs.

Evidence check. Preserve an untouched test set; report confusion matrix, threshold-dependent metrics, calibration, segments, and uncertainty.

Quick action. `train -> validate/CV -> choose threshold -> test once`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 078 | WEEK 10

Introduction to Week 10 Convolutional Neural Networks (CNNs)

Core idea. Place this lecture in the learning journey and connect its vocabulary to an artifact you will build.

Evidence check. Write the prerequisite, expected output, one question, and the next lecture that answers it.

Quick action. `concept -> example -> artifact -> evidence`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 079 | WEEK 10

Day 1: Introduction to Convolutional Neural Networks

Core idea. Exploit local spatial structure with shared filters, nonlinearities, downsampling, and a controlled augmentation policy.

Evidence check. Trace tensor shapes and receptive field; visualize errors by class; report augmentation, normalization, latency, and parameter count.

Quick action. $N, C, H, W \rightarrow \text{conv} \rightarrow \text{activation} \rightarrow \text{pool} \rightarrow \text{head}$

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 080 | WEEK 10

Day 2: Convolutional Layers and Filters

Core idea. Exploit local spatial structure with shared filters, nonlinearities, downsampling, and a controlled augmentation policy.

Evidence check. Trace tensor shapes and receptive field; visualize errors by class; report augmentation, normalization, latency, and parameter count.

Quick action. $N, C, H, W \rightarrow \text{conv} \rightarrow \text{activation} \rightarrow \text{pool} \rightarrow \text{head}$

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 081 | WEEK 10

Day 3: Pooling Layers and Dimensionality Reduction

Core idea. Exploit local spatial structure with shared filters, nonlinearities, downsampling, and a controlled augmentation policy.

Evidence check. Trace tensor shapes and receptive field; visualize errors by class; report augmentation, normalization, latency, and parameter count.

Quick action. $N, C, H, W \rightarrow \text{conv} \rightarrow \text{activation} \rightarrow \text{pool} \rightarrow \text{head}$

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 082 | WEEK 10

Day 4: Building CNN Architectures with Keras and TensorFlow

Core idea. Exploit local spatial structure with shared filters, nonlinearities, downsampling, and a controlled augmentation policy.

Evidence check. Trace tensor shapes and receptive field; visualize errors by class; report augmentation, normalization, latency, and parameter count.

Quick action. $N, C, H, W \rightarrow \text{conv} \rightarrow \text{activation} \rightarrow \text{pool} \rightarrow \text{head}$

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 083 | WEEK 10

Day 5: Building CNN Architectures with PyTorch

Core idea. Exploit local spatial structure with shared filters, nonlinearities, downsampling, and a controlled augmentation policy.

Evidence check. Trace tensor shapes and receptive field; visualize errors by class; report augmentation, normalization, latency, and parameter count.

Quick action. $N, C, H, W \rightarrow \text{conv} \rightarrow \text{activation} \rightarrow \text{pool} \rightarrow \text{head}$

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 084 | WEEK 10

Day 6: Regularization and Data Augmentation for CNNs

Core idea. Exploit local spatial structure with shared filters, nonlinearities, downsampling, and a controlled augmentation policy.

Evidence check. Trace tensor shapes and receptive field; visualize errors by class; report augmentation, normalization, latency, and parameter count.

Quick action. $N, C, H, W \rightarrow \text{conv} \rightarrow \text{activation} \rightarrow \text{pool} \rightarrow \text{head}$

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 085 | WEEK 10

Day 7: CNN Project - Image Classification on Fashion MNIST or CIFAR-10

Core idea. Learn a mapping from labeled examples and judge it with a split strategy and metric that reflect deployment costs.

Evidence check. Preserve an untouched test set; report confusion matrix, threshold-dependent metrics, calibration, segments, and uncertainty.

Quick action. `train -> validate/CV -> choose threshold -> test once`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 086 | WEEK 11

Introduction to Week 11 Recurrent Neural Networks (RNNs) and Sequence Modeling

Core idea. Place this lecture in the learning journey and connect its vocabulary to an artifact you will build.

Evidence check. Write the prerequisite, expected output, one question, and the next lecture that answers it.

Quick action. `concept -> example -> artifact -> evidence`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 087 | WEEK 11

Day 1: Introduction to Sequence Modeling and RNNs

Core idea. Represent ordered data with recurrent state while controlling padding, hidden-state boundaries, and vanishing or exploding gradients.

Evidence check. Verify sequence order, mask and padding, truncation length, hidden-state reset, gradient norm, and evaluation on long examples.

Quick action. `tokens -> embeddings -> recurrent state -> task head`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 088 | WEEK 11

Day 2: Understanding RNN Architecture and Backpropagation Through Time (BPTT)

Core idea. Represent ordered data with recurrent state while controlling padding, hidden-state boundaries, and vanishing or exploding gradients.

Evidence check. Verify sequence order, mask and padding, truncation length, hidden-state reset, gradient norm, and evaluation on long examples.

Quick action. `tokens -> embeddings -> recurrent state -> task head`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 089 | WEEK 11

Day 3: Long Short-Term Memory (LSTM) Networks

Core idea. Represent ordered data with recurrent state while controlling padding, hidden-state boundaries, and vanishing or exploding gradients.

Evidence check. Verify sequence order, mask and padding, truncation length, hidden-state reset, gradient norm, and evaluation on long examples.

Quick action. `tokens -> embeddings -> recurrent state -> task head`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 090 | WEEK 11

Day 4: Gated Recurrent Units (GRUs)

Core idea. Represent ordered data with recurrent state while controlling padding, hidden-state boundaries, and vanishing or exploding gradients.

Evidence check. Verify sequence order, mask and padding, truncation length, hidden-state reset, gradient norm, and evaluation on long examples.

Quick action. `tokens -> embeddings -> recurrent state -> task head`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 091 | WEEK 11

Day 5: Text Preprocessing and Word Embeddings for RNNs

Core idea. Represent ordered data with recurrent state while controlling padding, hidden-state boundaries, and vanishing or exploding gradients.

Evidence check. Verify sequence order, mask and padding, truncation length, hidden-state reset, gradient norm, and evaluation on long examples.

Quick action. tokens -> embeddings -> recurrent state -> task head

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 092 | WEEK 11

Day 6: Sequence-to-Sequence Models and Applications

Core idea. Represent ordered data with recurrent state while controlling padding, hidden-state boundaries, and vanishing or exploding gradients.

Evidence check. Verify sequence order, mask and padding, truncation length, hidden-state reset, gradient norm, and evaluation on long examples.

Quick action. tokens -> embeddings -> recurrent state -> task head

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 093 | WEEK 11

Day 7: RNN Project - Text Generation or Sentiment Analysis

Core idea. Represent ordered data with recurrent state while controlling padding, hidden-state boundaries, and vanishing or exploding gradients.

Evidence check. Verify sequence order, mask and padding, truncation length, hidden-state reset, gradient norm, and evaluation on long examples.

Quick action. tokens -> embeddings -> recurrent state -> task head

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 094 | WEEK 12

Introduction to Week 12 Transformers and Attention Mechanisms

Core idea. Place this lecture in the learning journey and connect its vocabulary to an artifact you will build.

Evidence check. Write the prerequisite, expected output, one question, and the next lecture that answers it.

Quick action. concept -> example -> artifact -> evidence

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 095 | WEEK 12

Day 1: Introduction to Attention Mechanisms

Core idea. Use attention to mix token information while preserving order through positional signals and repeated residual blocks.

Evidence check. Track tokenization, mask semantics, sequence length, hidden shapes, task metric, latency, memory, and failure cases.

Quick action. $QK^T/\sqrt{d}$ -> mask -> softmax -> V

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 096 | WEEK 12

Day 2: Introduction to Transformers Architecture

Core idea. Use attention to mix token information while preserving order through positional signals and repeated residual blocks.

Evidence check. Track tokenization, mask semantics, sequence length, hidden shapes, task metric, latency, memory, and failure cases.

Quick action. $QK^T/\sqrt{d}$ -> mask -> softmax -> V

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 097 | WEEK 12

Day 3: Self-Attention and Multi-Head Attention in Transformers

Core idea. Use attention to mix token information while preserving order through positional signals and repeated residual blocks.

Evidence check. Track tokenization, mask semantics, sequence length, hidden shapes, task metric, latency, memory, and failure cases.

Quick action. $QK^T/\sqrt{d}$ $\rightarrow$ mask $\rightarrow$ softmax $\rightarrow$ V

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 098 | WEEK 12

Day 4: Positional Encoding and Feed-Forward Networks

Core idea. Use attention to mix token information while preserving order through positional signals and repeated residual blocks.

Evidence check. Track tokenization, mask semantics, sequence length, hidden shapes, task metric, latency, memory, and failure cases.

Quick action. $QK^T/\sqrt{d}$ $\rightarrow$ mask $\rightarrow$ softmax $\rightarrow$ V

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 099 | WEEK 12

Day 5: Hands-On with Pre-Trained Transformers - BERT and GPT

Core idea. Use attention to mix token information while preserving order through positional signals and repeated residual blocks.

Evidence check. Track tokenization, mask semantics, sequence length, hidden shapes, task metric, latency, memory, and failure cases.

Quick action. $QK^T/\sqrt{d}$ $\rightarrow$ mask $\rightarrow$ softmax $\rightarrow$ V

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 100 | WEEK 12

Day 6: Advanced Transformers - BERT Variants and GPT-3

Core idea. Use attention to mix token information while preserving order through positional signals and repeated residual blocks.

Evidence check. Track tokenization, mask semantics, sequence length, hidden shapes, task metric, latency, memory, and failure cases.

Quick action. $QK^T/\sqrt{d}$ -> mask -> softmax -> V

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 101 | WEEK 12

Day 7: Transformer Project - Text Summarization or Translation

Core idea. Use attention to mix token information while preserving order through positional signals and repeated residual blocks.

Evidence check. Track tokenization, mask semantics, sequence length, hidden shapes, task metric, latency, memory, and failure cases.

Quick action. $QK^T/\sqrt{d}$ -> mask -> softmax -> V

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 102 | WEEK 13

Introduction to Week 13 Transfer Learning and Fine-Tuning

Core idea. Place this lecture in the learning journey and connect its vocabulary to an artifact you will build.

Evidence check. Write the prerequisite, expected output, one question, and the next lecture that answers it.

Quick action. concept -> example -> artifact -> evidence

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 103 | WEEK 13

Day 1: Introduction to Transfer Learning

Core idea. Reuse learned representations, then adapt the head or selected layers with a controlled learning rate and domain-aware validation.

Evidence check. Record base checkpoint, frozen parameters, learning rates, unfreezing schedule, domain shift, forgetting, and comparison with scratch.

Quick action. `load -> freeze -> train head -> unfreeze selectively -> validate`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 104 | WEEK 13

Day 2: Transfer Learning in Computer Vision

Core idea. Reuse learned representations, then adapt the head or selected layers with a controlled learning rate and domain-aware validation.

Evidence check. Record base checkpoint, frozen parameters, learning rates, unfreezing schedule, domain shift, forgetting, and comparison with scratch.

Quick action. `load -> freeze -> train head -> unfreeze selectively -> validate`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 105 | WEEK 13

Day 3: Fine-Tuning Techniques in Computer Vision

Core idea. Reuse learned representations, then adapt the head or selected layers with a controlled learning rate and domain-aware validation.

Evidence check. Record base checkpoint, frozen parameters, learning rates, unfreezing schedule, domain shift, forgetting, and comparison with scratch.

Quick action. `load -> freeze -> train head -> unfreeze selectively -> validate`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 106 | WEEK 13

Day 4: Transfer Learning in NLP

Core idea. Reuse learned representations, then adapt the head or selected layers with a controlled learning rate and domain-aware validation.

Evidence check. Record base checkpoint, frozen parameters, learning rates, unfreezing schedule, domain shift, forgetting, and comparison with scratch.

Quick action. `load -> freeze -> train head -> unfreeze selectively -> validate`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 107 | WEEK 13

Day 5: Fine-Tuning Techniques in NLP

Core idea. Reuse learned representations, then adapt the head or selected layers with a controlled learning rate and domain-aware validation.

Evidence check. Record base checkpoint, frozen parameters, learning rates, unfreezing schedule, domain shift, forgetting, and comparison with scratch.

Quick action. `load -> freeze -> train head -> unfreeze selectively -> validate`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 108 | WEEK 13

Day 6: Domain Adaptation and Transfer Learning Challenges

Core idea. Reuse learned representations, then adapt the head or selected layers with a controlled learning rate and domain-aware validation.

Evidence check. Record base checkpoint, frozen parameters, learning rates, unfreezing schedule, domain shift, forgetting, and comparison with scratch.

Quick action. `load -> freeze -> train head -> unfreeze selectively -> validate`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 109 | WEEK 13

Day 7: Transfer Learning Project - Fine-Tuning for a Custom Task

Core idea. Reuse learned representations, then adapt the head or selected layers with a controlled learning rate and domain-aware validation.

Evidence check. Record base checkpoint, frozen parameters, learning rates, unfreezing schedule, domain shift, forgetting, and comparison with scratch.

Quick action. `load -> freeze -> train head -> unfreeze selectively -> validate`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 110 | WEEK 14

Introduction to Week 14 MLOps and Model Deployment

Core idea. Place this lecture in the learning journey and connect its vocabulary to an artifact you will build.

Evidence check. Write the prerequisite, expected output, one question, and the next lecture that answers it.

Quick action. `concept -> example -> artifact -> evidence`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 111 | WEEK 14

Day 1: Introduction to MLOps - Why It Matters

Core idea. Ship a versioned model behind a tested contract with reproducible inputs, automated gates, telemetry, and rollback.

Evidence check. Capture code/data/model versions, dependency lock, schema, smoke and load tests, health checks, metrics, security, and rollback.

Quick action. `build -> test -> package -> deploy -> verify -> observe -> rollback`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 112 | WEEK 14

Day 2: Version Control for Data and Models (Git, DVC)

Core idea. Ship a versioned model behind a tested contract with reproducible inputs, automated gates, telemetry, and rollback.

Evidence check. Capture code/data/model versions, dependency lock, schema, smoke and load tests, health checks, metrics, security, and rollback.

Quick action. `build -> test -> package -> deploy -> verify -> observe -> rollback`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 113 | WEEK 14

Day 3: Model Packaging with Pickle, ONNX, and TorchScript

Core idea. Ship a versioned model behind a tested contract with reproducible inputs, automated gates, telemetry, and rollback.

Evidence check. Capture code/data/model versions, dependency lock, schema, smoke and load tests, health checks, metrics, security, and rollback.

Quick action. `build -> test -> package -> deploy -> verify -> observe -> rollback`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 114 | WEEK 14

Day 4: Serving Models with Flask, FastAPI, and Streamlit

Core idea. Ship a versioned model behind a tested contract with reproducible inputs, automated gates, telemetry, and rollback.

Evidence check. Capture code/data/model versions, dependency lock, schema, smoke and load tests, health checks, metrics, security, and rollback.

Quick action. `build -> test -> package -> deploy -> verify -> observe -> rollback`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 115 | WEEK 14

Day 5: Deploying ML Models on AWS, GCP, and Azure

Core idea. Ship a versioned model behind a tested contract with reproducible inputs, automated gates, telemetry, and rollback.

Evidence check. Capture code/data/model versions, dependency lock, schema, smoke and load tests, health checks, metrics, security, and rollback.

Quick action. `build -> test -> package -> deploy -> verify -> observe -> rollback`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 116 | WEEK 14

Day 6: CI/CD Pipelines for Machine Learning Projects

Core idea. Ship a versioned model behind a tested contract with reproducible inputs, automated gates, telemetry, and rollback.

Evidence check. Capture code/data/model versions, dependency lock, schema, smoke and load tests, health checks, metrics, security, and rollback.

Quick action. `build -> test -> package -> deploy -> verify -> observe -> rollback`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 117 | WEEK 14

Day 7: MLOps Project - Build and Deploy a Model End-to-End

Core idea. Ship a versioned model behind a tested contract with reproducible inputs, automated gates, telemetry, and rollback.

Evidence check. Capture code/data/model versions, dependency lock, schema, smoke and load tests, health checks, metrics, security, and rollback.

Quick action. `build -> test -> package -> deploy -> verify -> observe -> rollback`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 118 | WEEK 15

Introduction to Week 15 - Generative AI and Large Language Model Applications

Core idea. Place this lecture in the learning journey and connect its vocabulary to an artifact you will build.

Evidence check. Write the prerequisite, expected output, one question, and the next lecture that answers it.

Quick action. `concept -> example -> artifact -> evidence`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 119 | WEEK 15

Day 1: Introduction to Generative AI and LLMs

Core idea. Build provider calls behind a narrow adapter and treat model output as untrusted data requiring validation, evaluation, and policy controls.

Evidence check. Keep keys server-side; pin provider/model metadata; add timeout, retry, rate limits, schema checks, logging, evaluations, and cost limits.

Quick action. `request -> provider adapter -> validate -> evaluate -> return`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 120 | WEEK 15

Day 2: Understanding Prompt Engineering and Tokenization

Core idea. Treat instructions, examples, context, output schema, and sampling as versioned inputs to an evaluated model call.

Evidence check. Test representative and adversarial cases; log prompt version, model, tokens, latency, cost, schema validity, and quality rubric.

Quick action. `system + task + context + constraints + output schema`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 121 | WEEK 15

Day 3: Building Applications with OpenAI, Gemini, and Claude APIs

Core idea. Build provider calls behind a narrow adapter and treat model output as untrusted data requiring validation, evaluation, and policy controls.

Evidence check. Keep keys server-side; pin provider/model metadata; add timeout, retry, rate limits, schema checks, logging, evaluations, and cost limits.

Quick action. `request -> provider adapter -> validate -> evaluate -> return`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 122 | WEEK 15

Day 4: Fine-Tuning and Custom Instruction Models

Core idea. Reuse learned representations, then adapt the head or selected layers with a controlled learning rate and domain-aware validation.

Evidence check. Record base checkpoint, frozen parameters, learning rates, unfreezing schedule, domain shift, forgetting, and comparison with scratch.

Quick action. `load -> freeze -> train head -> unfreeze selectively -> validate`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 123 | WEEK 15

Day 5: Building RAG (Retrieval-Augmented Generation) Pipelines

Core idea. Separate ingestion, chunking, embedding, retrieval, context assembly, generation, citation, and evaluation into measurable stages.

Evidence check. Evaluate retrieval recall and relevance separately from answer groundedness; log source IDs, token use, latency, cost, and failures.

Quick action. `question -> retrieve -> rerank -> context -> generate -> cite`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 124 | WEEK 15

Day 6: AI Agents and Autonomous Systems Overview

Core idea. Build provider calls behind a narrow adapter and treat model output as untrusted data requiring validation, evaluation, and policy controls.

Evidence check. Keep keys server-side; pin provider/model metadata; add timeout, retry, rate limits, schema checks, logging, evaluations, and cost limits.

Quick action. `request -> provider adapter -> validate -> evaluate -> return`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

LECTURE 125 | WEEK 15

Day 7: Generative AI Project - Build Your Own Chatbot or Content Generator

Core idea. Build provider calls behind a narrow adapter and treat model output as untrusted data requiring validation, evaluation, and policy controls.

Evidence check. Keep keys server-side; pin provider/model metadata; add timeout, retry, rate limits, schema checks, logging, evaluations, and cost limits.

Quick action. `request -> provider adapter -> validate -> evaluate -> return`

Exit prompt. Explain the concept, implement the smallest proof, name one failure mode, and defend the metric.

COVERAGE AUDIT

This reference mirrors the supplied course order exactly.

Scope	Count
Course-introduction lectures	5
Weekly lectures	120
Total lecture cards	125
Course weeks	15